

AR Vision

1. Experiment Objective

Use OpenCV to implement an augmented reality effect — locate the camera pose via a checkerboard and overlay 3D virtual objects onto the camera image.

2. Experiment Principle

1. Key Terms

Term	Description
Augmented Reality (AR)	Overlays computer-generated virtual objects onto the real camera image to achieve a fusion of the virtual and the real
Camera Intrinsics	A parameter matrix describing the camera's optical characteristics (focal length, principal point coordinates, distortion coefficients), used for 3D→2D projection
Checkerboard Calibration	Estimates the camera intrinsics and distortion coefficients by photographing a checkerboard of known dimensions and using corner detection and the PnP algorithm
PnP	The algorithm that solves the camera pose (rotation + translation) from correspondences between 2D image points and 3D space points
Corner Detection	<code>findChessboardCorners</code> + <code>cornersSubPix</code> sub-pixel refinement to locate the corners where black and white squares of the checkerboard meet

2. Technical Architecture

```
Terminal 1: start agent + camera + joint models
Terminal 2: start AR vision
```

Data Flow: ESP32 camera → image subscription → grayscale conversion → `findChessboardCorners` detects the 6×9 checkerboard corners → `solvePnP` solves the camera pose → pre-defined 3D model points are projected to the 2D image via the camera intrinsics → `cv2.line` / `cv2.fillPoly` draw the 3D shapes → real-time display. Key presses switch between 12 kinds of 3D shapes.

3. Core Topic Reference

Firmware Uplink (sensor data)

Topic	Message Type	QoS	Description
<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	best_effort + keep_last(1)	Raw image from the ESP32 camera; the input source for checkerboard detection

ROS2 Internal Topics

Topic	Message Type	QoS	Direction	Description
<code>/simpleAR/camera</code>	<code>sensor_msgs/Image</code>	reliable	Publish	Output image with the 3D shapes overlaid after AR processing
<code>/Graphics_topic</code>	<code>std_msgs/String</code>	reliable	Subscribe	Remote shape-switching command; publish a shape name (e.g. <code>"Ball"</code>) to switch

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ Version	☒ Supported
No-PTZ Version	☒ Supported
Required Peripherals	ESP32 camera, checkerboard calibration board (factory-calibrated, ready to use)

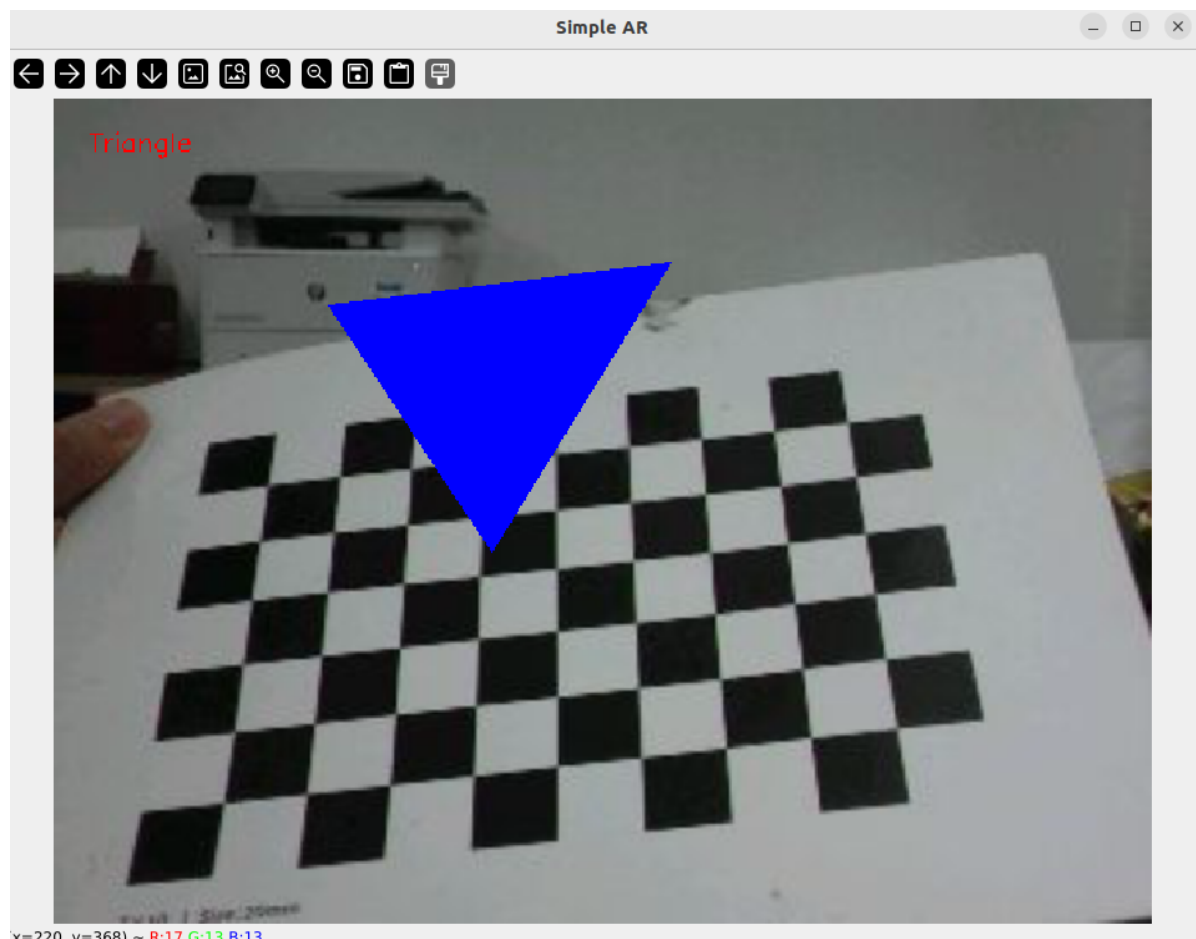
2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment can1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment can2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046238399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start AR Vision (Terminal 2)

```
ros2 launch microros_v2_opencv_visual_pkg simple_ar.launch.py
```



4. Operation Instructions

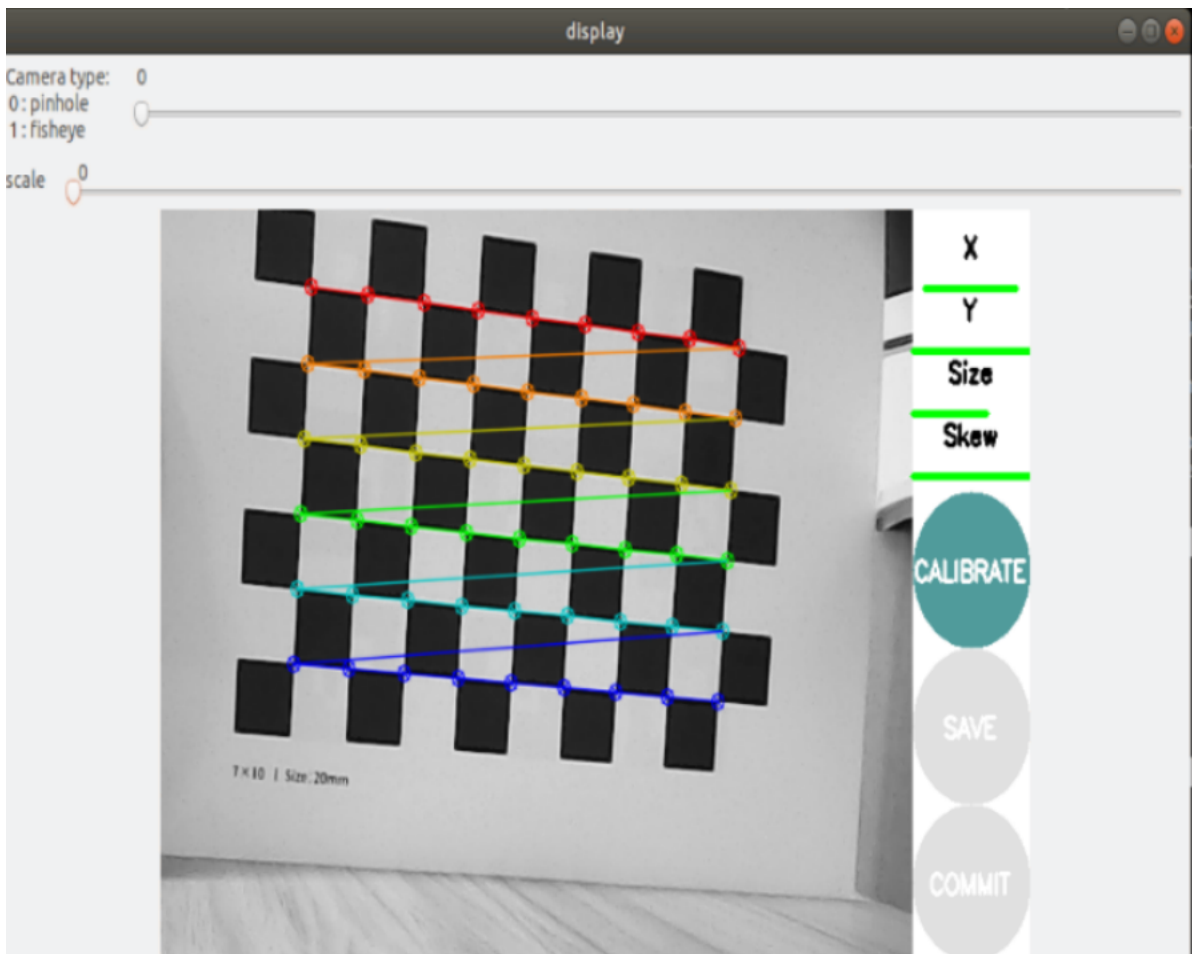
Workflow: Grayscale conversion → `findChessboardCorners` detects the 6×9 checkerboard → `cornerSubPix` sub-pixel refinement → `solvePnP` solves the pose → the point sets of the 12 3D shapes are projected to the 2D image via the camera intrinsics → draw lines and polygons.

Key	Function
f / F	Cycle forward to the next 3D virtual shape (12 shapes: Triangle → Rectangle → ... → ParallelBars → Triangle)
q	Exit the node

Remote shape switching: The node also subscribes to `/Graphics_topic` (`std_msgs/String`). You can switch shapes by directly specifying a shape name through a ROS2 topic, for example:

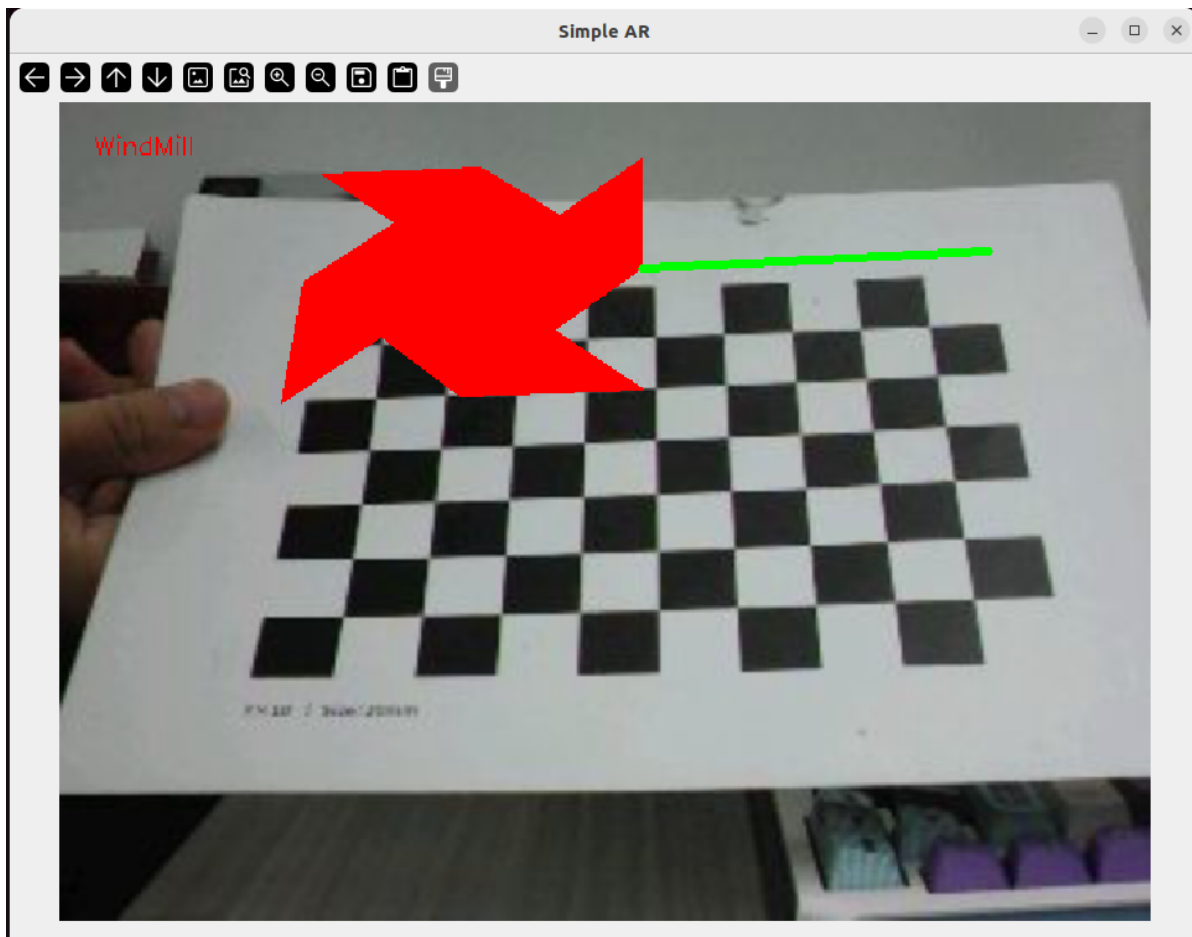
```
ros2 topic pub /Graphics_topic std_msgs/msg/String "data: 'Ball'"
```

The camera intrinsics are factory-calibrated and stored in `camera.yaml`.



4. Experiment Observations

- Place the checkerboard in the camera's field of view; colored 3D virtual shapes are overlaid and follow the checkerboard pose in real time
- Press **f / F** to cycle through the 12 different 3D shapes in order



5. Program Analysis

Source code paths:

- Launch:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/launch/simple_ar.launch.py`
- Node:
`~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/simple_ar.py`

1. Core Node Analysis

`SimpleARNode` locates the camera pose through checkerboard detection, projects the pre-defined 3D model points to the 2D image, and draws them.

```
# Source file:
~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/simple_ar.py
class SimpleARNode(Node):
    def __init__(self):
        super().__init__('simple_ar_node')
        self.bridge = cvBridge()

    # OpenCV display window configuration
    self.window_name = 'Simple AR'
    self.display_width = 960
    self.display_height = 720
    cv.namedWindow(self.window_name, cv.WINDOW_NORMAL)
    cv.resizeWindow(self.window_name, self.display_width,
self.display_height)
```

```

# Publish the AR-processed image
self.pub_img = self.create_publisher(Image, '/simpleAR/camera', 1)

# Subscribe to the shape switching topic
self.sub_graphics = self.create_subscription(
    String, '/Graphics_topic', self.choose_graphics, 1)

# Subscribe to the raw image topic
self.subscription = self.create_subscription(
    Image, '/camera/image_raw',
    self.image_callback,
    QoSProfile(history=QoSHistoryPolicy.KEEP_LAST, depth=1,
                reliability=QoSReliabilityPolicy.BEST_EFFORT,
                durability=QSDurabilityPolicy.VOLATILE))

# Current shape index and mirror flag
self.index = 0
self.mirror = False

# Predefined vertex sets of the 12 3D shapes (partial examples)
self.graphics = [
    "Triangle", "Rectangle", "Parallelogram", "WindMill",
    "TableTennisTable", "Ball", "Arrow", "Knife",
    "Desk", "Bench", "Stickman", "ParallelBars"
]
self.Graphics = self.graphics[self.index]

# Checkerboard inner corner specification 6x9
self.patternSize = (6, 9)

# 3D model vertex coordinates (78 points define the vertices and edges
of the 12 shapes)
self.axis = np.float32([
    [0, 0, -1], [0, 8, -1], [5, 8, -1], [5, 0, -1],
    [1, 2, -1], [1, 6, -1], [4, 2, -1], [4, 6, -1],
    # ... 78 3D coordinate points in total
])

# Checkerboard 3D world coordinates (6x9=54 corners)
self.objectPoints = np.zeros((6 * 9, 3), np.float32)
self.objectPoints[:, :2] = np.mgrid[0:6, 0:9].T.reshape(-1, 2)

# Load camera intrinsics (from camera.yaml)
config_path = (
    '~/microros_ws/src/'
    'microros_v2_opencv_visual_pkg/config'
)
yaml_path = f'{config_path}/camera.yaml'
if os.path.exists(yaml_path):
    with open(yaml_path, 'r') as f:
        calib_data = yaml.safe_load(f)
    self.cameraMatrix = np.array(
        calib_data['camera_matrix']['data'],
        dtype=np.float64).reshape(3, 3)
    self.distCoeffs = np.array(
        calib_data['distortion_coefficients']['data'],
        dtype=np.float64).reshape(1, -1)

```

```

        self.camera_ready = True

    def image_callback(self, msg):
        """Image callback: image conversion → checkerboard detection → PnP pose
        estimation → 3D projection → drawing"""
        try:
            frame = self.bridge.imgmsg_to_cv2(
                msg, desired_encoding='bgr8')
        except Exception as e:
            self.get_logger().error(f'Image decode failed: {e}')
            return

        if self.mirror:
            frame = cv.flip(frame, 1)

        action = cv.waitKey(10) & 0xFF
        frame = self.process(frame, action)
        cv.imshow(self.window_name, frame)

        if action == ord('q') or action == 113:
            self.get_logger().info('Exiting the application...')
            rclpy.shutdown()

    def process(self, img, action):
        """Core processing pipeline: key switching → grayscale → corner
        detection → sub-pixel refinement → PnP → 3D projection"""
        if action == ord('f') or action == ord('F'):
            self.graphics_update()

        if not self.camera_ready:
            return img # show the original image only when the camera is not
            calibrated

        gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)

        # Checkerboard corner detection (6x9 inner corners, with adaptive
        threshold + fast detection flags)
        retval, corners = cv.findChessboardCorners(
            gray, self.patternSize, None,
            flags=cv.CALIB_CB_ADAPTIVE_THRESH
            + cv.CALIB_CB_NORMALIZE_IMAGE
            + cv.CALIB_CB_FAST_CHECK)

        if retval:
            # Sub-pixel corner refinement (30 iterations, precision 0.001)
            corners = cv.cornerSubPix(
                gray, corners, (11, 11), (-1, -1),
                (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001))

            # PnP + RANSAC robust pose estimation
            retval, rvec, tvec, inliers = cv.solvePnPRansac(
                self.objectPoints, corners,
                self.cameraMatrix, self.distCoeffs)

            if retval:
                # Project the 78 3D model points onto the 2D image
                image_points, jacobian = cv.projectPoints(
                    self.axis, rvec, tvec,

```

```

        self.cameraMatrix, self.distCoeffs)
    # Draw according to the currently selected shape
    img = self.draw(img, corners, image_points)

    self.pub_img.publish(self.bridge.cv2_to_imgmsg(img, encoding='bgr8'))
    return img

```

Key logic notes:

- **Checkerboard detection:** `findChessboardCorners` uses the `CALIB_CB_ADAPTIVE_THRESH` + `CALIB_CB_FAST_CHECK` flags for adaptive thresholding and fast detection, which suits real-time AR scenarios
- **Sub-pixel refinement:** `cornerSubPix` improves corner coordinate accuracy to the sub-pixel level (0.001 px), reducing pose estimation error
- **PnP + RANSAC:** `solvePnP` uses the RANSAC algorithm for robust pose estimation, tolerating some misdetections or occluded corners
- **3D→2D projection:** `projectPoints` projects the 78 pre-defined 3D vertices onto the 2D plane using the camera intrinsics and pose — the core operation of AR
- **12 shapes:** the `draw()` method uses `cv.drawContours` / `cv.line` / `cv.circle` to draw the 2D projected outlines of the different 3D shapes based on the current value of `self.Graphics`

2. Key Parameters

Source code path:

```

~/microros_ws/src/microros_v2_opencv_visual_pkg/microros_v2_opencv_visual_pkg/
simple_ar.py

```

This node declares no ROS2 parameters (`simple_ar.launch.py` has no `DeclareLaunchArgument`, and the source has no `self.declare_parameter()`); the core configuration consists of hard-coded constants:

Constant	Value	Location	Description
<code>patternSize</code>	(6, 9)	<code>__init__</code> L90	Number of checkerboard inner corners (columns × rows)
<code>axis</code>	78×3 coordinate array	<code>__init__</code> L92-113	Vertex coordinates of the 12 3D shapes
<code>graphics</code>	12 strings	<code>__init__</code> L74-87	Shape name list (Triangle ~ ParallelBars)
<code>yaml_path</code>	<code>.../config/camera.yaml</code>	<code>__init__</code> L119-124	Camera intrinsic file path (hard-coded)

To modify the checkerboard specification or the camera calibration file, you need to edit the corresponding lines directly in the source code.

3. Node Description

Node	Function
<code>simple_ar_node</code>	Checkerboard detection + PnP pose estimation + 3D shape projection + key-based switching between 12 shapes

6. Frequently Asked Questions

Problem	Cause	Solution
Checkerboard not detected	Uneven lighting or too large an angle	Adjust the checkerboard angle and ensure uniform lighting
3D shapes jitter	PnP pose estimation noise	Keep the checkerboard steady in the frame and avoid fast movement